

LiquidAI / LFM2.5-Co1BERT-350M

like

107

Follow

Liquid AI

4.34k



Sentence Similarity



Safetensors



sentence-transformers



11 languages

PyLate

lfm2

liquid

lfm2.5

edge

Co1BERT

feature-extraction

custom_code

arxiv:2511.23404



License: lfm1.0



→ Copy to bucket

NEW

Use this model



Model card



Files



xet



Community

2


[Try LFM](#) • [Docs](#) • [LEAP](#) • [Discord](#)

LFM2.5-Co1BERT-350M

We release two new **best-in-class multilingual retrieval** models:

- [LFM2.5-Embedding-350M](#) — A dense bi-encoder, one vector per document. Smallest, fastest index.
- **LFM2.5-Co1BERT-350M** — A late-interaction model. One vector per *token*, matched via MaxSim. Higher accuracy and better generalization at the cost of index size.

Both models are 350M params and the first bidirectional members of the LFM family, built on [LFM2.5-350M-Base](#). They can be used as a **drop-in replacement** for your current RAG pipeline and

Downloads last month

11,062


Safetensors



Model size

0.4B params

Tensor type

BF16



Chat template

↗ Files info

Inference Providers

NEW



Sentence Similarity

This model isn't deployed by any Inference Provider.



3

Ask for provider support

Model tree for LiquidAI/LFM2.5-C...

Base model [LiquidAI/LFM2.5-350M-Ba...](#)



Finetuned (14)

[this model](#)

Finetunes

11 models

Quantizations

3 models



Spaces using LiquidAI/LFM2.5-...

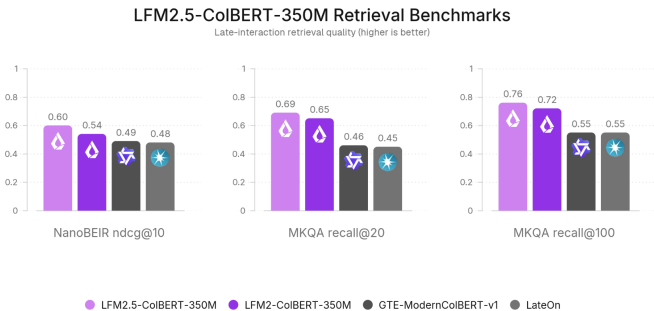
4

target fast, cheap, and reliable multilingual / cross-lingual search across 11 languages.

Find more details about the bidirectional architecture and training recipe in our [blog post](#).

 Demo:

<https://huggingface.co/spaces/LiquidAI/colbert-tool-selection>



 Model details

Property	LFM2.5-ColBERT-350M	<u>LFM2.5-Embedding-350M</u>
Type	Late interaction (per-token vectors)	Dense bi-encoder (single vector)
Total parameters	~353M	~354M
Backbone	<u>LFM2.5-350M-Base</u> + bi-directional patches	<u>LFM2.5-350M-Base</u> + bi-directional patches
Layers	17 (10 conv + 6 attn + 1 dense)	17 (10 conv + 6 attn + 1 pool)

 [LiquidAI/colbert-tool-selection](#)

 [mteb/leaderboard](#)

 [hakari-bench/leaderboard](#)

 [developerjeremylive/colbert-tool-s...](#)

 Collection including LiquidAI/LFM...

 **Liquid Nanos**  Collection
Library of tas... • 34 items • U.. •  124

 Paper for LiquidAI/LFM2.5-ColBE...

LFM2 Technical Report
 Paper • 2511.23404 • Publi... •  65

Property	LFM2.5-ColBERT-350M	<u>LFM2.5-Embedding-350M</u>
Vocabulary size	64,402	65,536
Output	128-dim per token	1024-dim CLS vector
Similarity	MaxSim	Cosine
Training precision	BF16	BF16
License	LFM Open License v1.0	LFM Open License v1.0

Document length: 512 tokens

Query length: 32 tokens

Supported languages: English, Spanish, German, French, Italian, Portuguese, Arabic, Swedish, Norwegian, Japanese, Korean.

Architecture:

```
ColBERT(  
  (0): Transformer({'max_seq_length': 511, '  
  (1): Dense({'in_features': 1024, 'out_featu  
)
```

We recommend LFM2.5-Embedding-350M and LFM2.5-ColBERT-350M for short-context retrieval use cases, such as:

- **E-commerce:** find products across many languages with semantic search at scale.

- **FAQ and support knowledge bases:** retrieve the right answer reliably across customer-facing surfaces.
- **On-device semantic search:** search files, emails, and notes locally on consumer hardware.
- **Enterprise knowledge assistants:** retrieve internal legal, financial, and technical documents across languages.



How to run



First, install the PyLate and transformers libraries:

```
pip install -U pylate
```

Retrieval

Use this model with PyLate to index and retrieve documents. The index uses [FastPLAID](#) for efficient similarity search.

Indexing documents

Load LFM2.5-ColBERT-350M and initialize the PLAID index, then encode and index your documents:

```
from pylate import indexes, models, retrieve

# Step 1: Load the ColBERT model (trust_remote
model = models.ColBERT(
    model_name_or_path="LiquidAI/LFM2.5-ColBI
    trust_remote_code=True,
)
```

```
model.tokenizer.pad_token = model.tokenizer.unk_token
```

```
# Step 2: Initialize the PLAID index
```

```
index = indexes.PLAID(  
    index_folder="pylate-index",  
    index_name="index",  
    override=True, # This overwrites the existing index  
)
```

```
# Step 3: Encode the documents
```

```
documents_ids = ["1", "2", "3"]  
documents = ["document 1 text", "document 2 text", "document 3 text"]
```

```
documents_embeddings = model.encode(  
    documents,  
    batch_size=32,  
    is_query=False, # Ensure that it is set to False  
    show_progress_bar=True,  
)
```

```
# Step 4: Add document embeddings to the index
```

```
index.add_documents(  
    documents_ids=documents_ids,  
    documents_embeddings=documents_embeddings,  
)
```

Note that you do not have to recreate the index and encode the documents every time. Once you have created an index and added the documents, you can re-use the index later by loading it:

```
# To load an index, simply instantiate it with the same parameters
```

```
index = indexes.PLAID(  
    index_folder="pylate-index",  
    index_name="index",  
)
```

Retrieving top-k documents for queries

Once the documents are indexed, you can retrieve the top-k most relevant documents for a given set of queries. To do so, initialize the ColBERT retriever with the index you want to search in, encode the queries, and then retrieve the top-k documents to get the top matches ids and relevance scores:

```
# Step 1: Initialize the ColBERT retriever
retriever = retrieve.ColBERT(index=index)

# Step 2: Encode the queries
queries_embeddings = model.encode(
    ["query for document 3", "query for document 4"],
    batch_size=32,
    is_query=True, # Ensure that it is set as a query
    show_progress_bar=True,
)

# Step 3: Retrieve top-k documents
scores = retriever.retrieve(
    queries_embeddings=queries_embeddings,
    k=10, # Retrieve the top 10 matches for each query
)
```

Reranking

If you only want to use LFM2.5-ColBERT-350M to perform reranking on top of your first-stage retrieval pipeline without building an index, you can simply use the `rank` function and pass the queries and documents to rerank:

```
from pylate import rank, models

queries = [
    "query A",
    "query B",
]
```

```

documents = [
    ["document A", "document B"],
    ["document 1", "document C", "document B"]
]

documents_ids = [
    [1, 2],
    [1, 3, 2],
]

model = models.ColBERT(
    model_name_or_path="LiquidAI/LFM2.5-ColBERT",
    trust_remote_code=True,
)

queries_embeddings = model.encode(
    queries,
    is_query=True,
)

documents_embeddings = model.encode(
    documents,
    is_query=False,
)

reranked_documents = rank.rerank(
    documents_ids=documents_ids,
    queries_embeddings=queries_embeddings,
    documents_embeddings=documents_embeddings,
)

```

Performance

We highlight (= bold) the best bi-encoder and best late retriever for each language.

NanoBEIR Multilingual Extended — NDCG@10

Multilingual retrieval capabilities.

Model	Type	AVG	ar	de	en
LiquidAI/LFM2.5-ColBERT-350M	late	0.605	0.551	0.606	0.6
LiquidAI/LFM2.5-Embedding-350M	dense	0.577	0.529	0.581	0.6
Qwen/Qwen3-Embedding-0.6B	dense	0.556	0.514	0.560	0.6
LiquidAI/LFM2-ColBERT-350M	late	0.540	0.491	0.563	0.6
Alibaba-NLP/gte-multilingual-base	dense	0.528	0.477	0.523	0.6
lightonai/GTE-ModernColBERT-v1	late	0.489	0.309	0.499	0.6
lightonai/LateOn	late	0.484	0.307	0.505	0.6
lightonai/DenseOn	dense	0.432	0.178	0.474	0.6
Alibaba-NLP/gte-modernbert-base	dense	0.383	0.112	0.449	0.6
BAAI/bge-large-en-v1.5	dense	0.359	0.059	0.419	0.6

MKQA-11 — Recall@20

MKQA. Cross-lingual capabilities (subset of the 11 languages we target).

Model	Type	AVG	ar	de	en
LiquidAI/LFM2.5-ColBERT-350M	late	0.694	0.608	0.709	0.7
LiquidAI/LFM2.5-Embedding-350M	dense	0.691	0.610	0.709	0.7
Alibaba-NLP/gte-multilingual-base	dense	0.675	0.567	0.692	0.7
LiquidAI/LFM2-ColBERT-350M	late	0.646	0.554	0.696	0.7
Qwen/Qwen3-Embedding-0.6B	dense	0.638	0.520	0.671	0.7
lightonai/GTE-ModernColBERT-v1	late	0.459	0.092	0.532	0.7
lightonai/LateOn	late	0.454	0.157	0.492	0.7
lightonai/DenseOn	dense	0.435	0.165	0.482	0.7
BAAI/bge-large-en-v1.5	dense	0.413	0.133	0.471	0.7
Alibaba-NLP/gte-modernbert-base	dense	0.295	0.060	0.333	0.7

Inference speed - llama.cpp

End-to-end latency on **MacBook Pro M4 Max** via **llama.cpp** at **fp16**, measured at **32-token queries** and **256-token documents**. Docs cached means that the document embeddings are pre-computed and looked up (from an index).

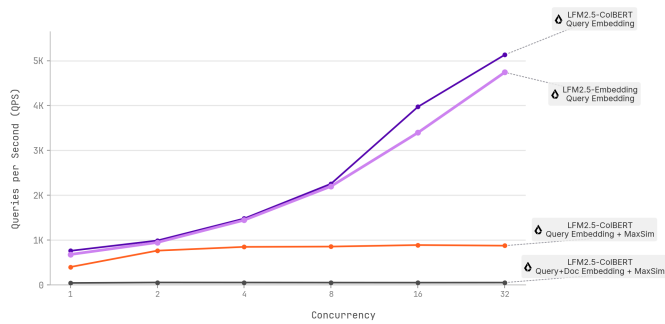
Model	Stage	Docs cached	p50	p95
LFM2.5- Embedding- 350M	Query embedding	yes	7.3 ms	9.6 ms
LFM2.5- ColBERT- 350M	Query embedding	yes	8.1 ms	8.5 ms
LFM2.5- ColBERT- 350M	Query embedding + MaxSim	yes	8.2 ms	15.2 ms
LFM2.5- ColBERT- 350M	Query embedding + Doc embedding + MaxSim	no	34.3 ms	36.3 ms

Both models [LiquidAI/LFM2.5-ColBERT-350M-GGUF](#) and [LiquidAI/LFM2.5-Embedding-350M-GGUF](#) are available on Hugging Face under different quantization schemas for llama.cpp.

Inference speed - Enterprise GPU

For large-scale production-grade enterprise deployments, we also experiment with an internal GPU stack to deliver extremely low-latency serving under high inbound load. We observe latencies as low as 1 ms.

LFM2.5 Retrieval Models
Internal GPU Stack QPS (single H100)



Workload	Set up	p50	p95	p99
LFM2.5-Embedding-350M	Query embedding	1.5 ms	1.6 ms	1.7 ms
LFM2.5-ColBERT-350M	Query embedding	1.3 ms	1.4 ms	1.5 ms
LFM2.5-ColBERT-350M	Query embedding + MaxSim	2.5 ms	2.7 ms	2.8 ms
LFM2.5-ColBERT-350M	Query embedding + Doc embedding + MaxSim	22.8 ms	24.1 ms	26.4 ms

Contact

- Got questions or want to connect? [Join our Discord community](#).
- If you are interested in custom solutions with edge deployment, please contact [our sales team](#).

Citation

@article{liquidai2025lfm2,

 System theme [TOS](#) [Privacy](#) [About](#) [Careers](#)



[Models](#)

[Datasets](#)

[Spaces](#)

[Pricing](#)

[Docs](#)

author={Liquid AI},

journal={arXiv preprint arXiv:2511.23404},

year={2025}

}

@misc{PyLate,

title={PyLate: Flexible Training and Retrieval},

author={Chaffin, Antoine and Sourty, Raphaël},

url={https://github.com/lightonai/pylate},

year={2024}

}